

Lab 5 Report: Localization

Team 4

Christian Cassamajor-Paul

Alicia Ramirez

Leo Sun

Raymond Wang

RSS

April 11, 2026

1 Introduction

A big part of autonomous navigation is localization, or, knowing the vehicle's location and heading before making meaningful planning or control decisions. In this lab, we designed a Monte Carlo Localization (MCL) system for the RACECAR platform that estimates the car's 2D pose in a known occupancy-grid map using noisy odometry and lidar measurements. This state-estimation layer acts as the connection between perception and planning/control. It turns raw motion and sensing data into a usable pose estimate, and supports the next stage of autonomous navigation where path planning relies on dependable localization. The purpose of the lab was to implement, tune, and evaluate a real-time particle filter in both simulation and on the physical car. At a high level, the technical challenge is that odometry accumulates drift, lidar returns are noisy, and different locations in the map can initially appear similar, so a single pose estimate is often unreliable. Our solution is to maintain many pose hypotheses, propagate them forward with a motion model, compare predicted and observed lidar ranges with a sensor model, and repeatedly reweight and resample the hypotheses so they converge to the most likely pose. This design lets the system stay robust to uncertainty while producing a fast, consistent localization estimate that downstream autonomous modules can trust.

2 Technical Approach

2.1 Particle Filter Initialization - Christian

Before any sort of motion model or sensor model updates can be made, we first initialize a particle cloud with N unique particles (using 2D pose estimate RVIZ button). To do this, we first take the provided input pose vector, which consists of an x coordinate, y coordinate, and quaternion, and convert it to a simpler form: a 3-vector of x, y, θ where θ is the rotation about the z-axis, aka yaw, of the racecar. Next, we generate N vectors that look like $[\epsilon_x, \epsilon_y, \epsilon_\theta]$, where the epsilons represent noise for their respective pose components for a different particle. Each value is sampled from a Gaussian distribution with a mean of 0 and a standard deviation of 0.1. We then add the simpler input pose to every noise vector to get N unique vectors that represent a cloud of potential poses centered around the provided input.

2.2 Motion Model - Christian

The motion model is responsible for updating the various pose hypotheses based on the latest odometry data. By injecting noise into the motion model, we aim to cover a range of possible updates that represent all the potential poses that the racecar may have.

On every odometry update after the cloud has been initialized, the motion model will apply the motion from the odometry to each particle, respectively, to that particle's frame, and then add further noise to each component of the pose to represent the noisiness and subsequent uncertainty of real-life odometry data. To be more specific, the following process occurs:

1. Odometry data received as a vector: $[\Delta x, \Delta y, \Delta \theta]$.
2. Use the pose info from timestep t-1 of each particle to transform and apply the odometry in the particle's frame for the current timestep t with the following equations:

$$x_t = x_{t-1} + \Delta x \cos(\theta_{t-1}) - \Delta y \sin(\theta_{t-1}) \quad (1)$$

$$y_t = y_{t-1} + \Delta x \sin(\theta_{t-1}) + \Delta y \cos(\theta_{t-1}) \quad (2)$$

$$\theta_t = \theta_{t-1} + \Delta \theta \quad (3)$$

3. Sample from a Gaussian distribution and then add noise for each particle in a similar manner to the initialization to ensure particles spread and cover a range of potential poses.

The Nx3 array of particles that results from the above process is returned to the particle filter and used in later operations as explained in Section 2.4.

2.3 Sensor Model - Ray and Alicia

The sensor model determines the probability of a particle, in the MCL pipeline, being in the correct location. This likelihood serves as the particle’s weight during resampling.

2.3.1 Beam Model

The beam model decomposes $p(z \mid d)$ —the probability of measuring range z when the true (ray-cast) range is d —into a mixture of four distributions:

$$p(z \mid d) = \alpha_{\text{hit}} p_{\text{hit}}(z \mid d) + \alpha_{\text{short}} p_{\text{short}}(z \mid d) + \alpha_{\text{max}} p_{\text{max}}(z) + \alpha_{\text{rand}} p_{\text{rand}}(z) \quad (4)$$

- **Hit** (p_{hit}): A Gaussian centered at the true range d :

$$p_{\text{hit}}(z \mid d) = \eta_{\text{hit}} \exp\left(-\frac{(z - d)^2}{2\sigma_{\text{hit}}^2}\right), \quad z \in [0, z_{\text{max}}] \quad (5)$$

- **Short** (p_{short}): Accounting for unexpected obstacles that return shorter-than-expected ranges:

$$p_{\text{short}}(z \mid d) = \frac{2}{d} \left(1 - \frac{z}{d}\right), \quad z \leq d \quad (6)$$

- **Max** (p_{max}): A point mass at z_{max} , representing complete sensor failures or highly reflective surfaces.
- **Rand** (p_{rand}): A uniform distribution $1/z_{\text{max}}$ over the full range, capturing random returns.

The mixing weights were tuned empirically to $\alpha_{\text{hit}} = 0.69$, $\alpha_{\text{short}} = 0.12$, $\alpha_{\text{max}} = 0.07$, $\alpha_{\text{rand}} = 0.12$, with $\sigma_{\text{hit}} = 0.5$ (in discretized pixel units). We have comparatively greater weight on α_{short} to trust the motion model more due to many non-mapped obstacles in Stata.

2.3.2 Precomputed Lookup Table

For efficiency we precomputed all values of Equation 4 into a table T , where entry $T[z, d]$ stores the column-normalized probability $p(z \mid d)$. During each update, measured and ray-cast ranges are converted from meters to integers for direct table indexing.

2.3.3 Particle Weight Computation Using Lookup Table

For particle $\mathbf{x}_i$, RangeLibc performs ray casting through the occupancy grid to produce expected ranges $\mathbf{d}_i = (d_i^1, \dots, d_i^B)$. Assuming conditional independence across beams, the log-likelihood is:

$$\log p(\mathbf{z}_k | \mathbf{x}_i, m) = \sum_{b=1}^B \log T[z_k^b, d_i^b] \quad (7)$$

To prevent the probabilities from collapsing on a few particles we compute a geometric mean:

$$w_i = \exp\left(\frac{1}{B} \sum_{b=1}^B \log T[z_k^b, d_i^b]\right) \quad (8)$$

This retains sensor information while still discriminating among poses

2.4 Particle Filter: Combining the Motion and Sensor Model

Our node separates the Bayes filter into an odometry-driven prediction step and a LiDAR-driven correction step. On each odometry callback, we compute a control increment

$$\mathbf{u}_t = \begin{bmatrix} v_x \Delta t \\ v_y \Delta t \\ \omega \Delta t \end{bmatrix}, \quad (9)$$

Each particle is then propagated through the motion model described earlier:

$$\mathbf{x}_t^{[i]} \sim p(\mathbf{x}_t | \mathbf{u}_t, \mathbf{x}_{t-1}^{[i]}). \quad (10)$$

This prediction step captures the short-term dynamics of the vehicle while allowing uncertainty to grow through injected Gaussian noise.

On each LiDAR callback, we downsample the incoming scan to $B = 100$ beams and evaluate the sensor-model likelihood from Equation 8 for every particle. The unnormalized particle weights are

$$\tilde{w}_t^{[i]} = p(\mathbf{z}_t | \mathbf{x}_t^{[i]}, m), \quad (11)$$

and the normalized weights are

$$w_t^{[i]} = \frac{\tilde{w}_t^{[i]}}{\sum_{j=1}^N \tilde{w}_t^{[j]}}. \quad (12)$$

Resampling every scan update causes fast convergence, but it also increases the risk of particle impoverishment. In our implementation, we resample stochastically on roughly half of the sensor updates rather than on every update. When

resampling is triggered, ancestor indices are drawn according to the normalized weights,

$$a_i \sim \text{Categorical}(w_t^{[1]}, \dots, w_t^{[N]}), \quad (13)$$

and each new particle is copied from its selected ancestor with a small Gaussian perturbation:

$$\mathbf{x}_t^{[i]} \leftarrow \mathbf{x}_t^{[a_i]} + \boldsymbol{\epsilon}_i, \quad \boldsymbol{\epsilon}_i \sim \mathcal{N}(\mathbf{0}, \text{diag}(\sigma_x^2, \sigma_y^2, \sigma_\theta^2)). \quad (14)$$

This preserved particle diversity better than resampling every frame. After each update, we compute a single pose estimate from the weighted particle set. The position is the weighted mean,

$$\hat{x}_t = \sum_{i=1}^N w_t^{[i]} x_t^{[i]}, \quad \hat{y}_t = \sum_{i=1}^N w_t^{[i]} y_t^{[i]}, \quad (15)$$

3 Experimental Evaluation

The purpose of this section is to **provide evidence of the functionality** of your design, and to **document your experimental evaluation**. The section should explain both:

1. **what** was tested and **why**, and **how** those tests were performed (Technical Procedures, including a clear definition of the performance metrics used in the analysis),
2. and **discuss the result** of those tests to arrive at an assessment of the functionalities you implemented in this lab (Results).

You can find ideas and suggestions in the “Good Experimental Evaluation” Recitation on Canvas (Modules section).

(up to 700 words)

3.1 Motion Model Evaluation - Christian

3.2 Localization Evaluation

4 Conclusion

In this lab, we designed, implemented, and evaluated a Monte Carlo Localization system for the RACECAR platform in both simulation and the real world. Our final system combined a noisy odometry-based motion model, a LiDAR beam-based sensor model, and a particle filter that maintained and updated a distribution of pose hypotheses over time. Together, these components allowed the car to estimate its position and heading in a known occupancy-grid map

despite drift in odometry, noise in LiDAR measurements, and ambiguity in the environment. We also adapted the system to run in real time by using vectorized operations, downsampling LiDAR beams, precomputing the sensor model table, and carefully balancing convergence against particle diversity during resampling.

Overall, this design phase produced a localization pipeline that is both functional and practical for autonomous driving tasks. Our experiments showed that the particle filter could recover from uncertainty, converge to the correct pose, and provide a stable map-frame estimate for downstream modules. At the same time, this lab made clear that localization performance depends strongly on parameter tuning, map quality, and environmental structure, especially on the physical car where dynamic obstacles and sensing imperfections are more pronounced than in simulation.

While the core localization system is now in place, there is still room for refinement. Future improvements could include more adaptive resampling strategies, better parameter tuning across different environments, and stronger robustness to highly dynamic scenes. Most importantly, this lab provides the state-estimation foundation needed for the next phase of the autonomy stack. With a dependable localization estimate available, the system is now prepared to build toward path planning and more complete autonomous navigation.

5 Lessons Learned

Christian Cassamajor-Paul

I learned that good documentation is necessary for keeping essential knowledge accessible to the team.

Alicia Ramirez

I learned that finishing the code early so that we have half a week for testing is very helpful, and that making a testing plan beforehand speeds everything up.

Leo Sun

I learned that rosbags can be used not only to collect data, but also to increase working efficiency.

Raymond Wang

I learned that modularized testing makes debugging and interpreting results much faster.